

Solving Sokoban with backward reinforcement learning

Yaron Shoham, Gal Elidan

Google research Israel
yaronsh,elidan@google.com

Abstract

In some puzzles, the strategy we need to use near the goal can be quite different from the strategy that is effective earlier on, e.g. due to a smaller branching factor near the exit state in a maze. A common approach in these cases is to apply both a forward and a backward search, and to try and align the two.

In this work we propose an approach that takes this idea a step forward, within a reinforcement learning (RL) framework. Training a traditional forward-looking agent using RL can be difficult because rewards are often sparse, e.g. given only at the goal. Instead, we first train a backward-looking agent with a simple relaxed goal. We then augment the state representation of the puzzle with straightforward *hint features* that are extracted from the behavior of that agent. Finally, we train a forward looking agent with this informed augmented state.

We demonstrate that this simple "access" to partial backward plans leads to a substantial performance boost. On the challenging domain of the Sokoban puzzle, our RL approach substantially surpasses the best learned solvers that generalize over levels, and is competitive with SOTA performance of the best highly-crafted solution. Impressively, we achieve these results while learning from only a small number of practice levels and using simple RL techniques.

Introduction

Coping with control and planning challenges in complex search spaces has been a core challenge of reinforcement learning (RL) for over four decades. In recent years, model free and (almost) feature free RL methods have gained wide popularity and have been able to, for example, surpass humans at playing Atari games while learning only from pixel data (Mnih et al. 2015) and score rewards, or defeat the world champion in the game of Go (Silver et al. 2016) while learning only from board positions and the single winning reward that comes at the end of a game. Such model-free methods are not sample efficient and, more often than not, require many orders of magnitude more experiences than a human would need to reach the same proficiency. This is particularly true when the rewards are sparse (e.g., given only at the end of the game), making it difficult for the RL agent to discover favorable states.

There are many settings where we do not have access and cannot simulate a large number of training trajectories, and are thus not able to make use of such model-free approaches.

In puzzle games, for example, we cannot pit one player vs. another thereby creating a de-facto infinite collection of training instances. In such cases, if we have a good model of the transition dynamics of the domain, we can sometimes use model-based RL to simulate the environment and look ahead. However, in complex settings, either the state space is too large to make such exploration practical, or we do not have access to such a transition model and training one is computationally infeasible.

In this work we focus on the challenging Japanese game of Sokoban where a player needs to push boxes into storage positions while facing the difficulty of obstacles. See Figure 1 for an example of a simple Sokoban level. Solution paths in Sokoban are much longer than Chess or Go, the agent faces the danger of irreversible moves (e.g. when a box is pushed into a corner), and the number of relevant training levels is limited. As a consequence, Sokoban remains a game where human performance far surpasses that of automated solvers.

As in many other domains, the reward in Sokoban is given only at a single state, upon placing all boxes at their designated targets. Intuitively, the only thing we can leverage in such scenarios is the fact that the target goal(s) is known and that we can somehow backtrack from this goal in order to assist the learning of the forward looking agent. Two recent works build on this intuition and rely on the idea of data augmentation or imitation, with the inherent assumption that the agent's behavior or policy near the goal is similar to the one at the beginning of the task (Edwards, Downs, and Davidson 2018; Goyal et al. 2019). However, such an assumption does not hold in general, and depends heavily on symmetric properties of the task at hand. (Florensa et al. 2017) suggest an interesting alternative for reverse learning that gradually increases the difficulty of the learning task by considering starting states that are further and further away from the goal state. The method relies on our ability to continue backtracking from any state, and is thus not applicable when it is easy to get stuck in deadlocked states. See the related work section for more details.

We suggest a simple approach that bypasses these assumptions. Instead of trying to imitate the policy of an agent that starts at the goal state and faces backwards, we use simple *hint features* of trajectories of that agent to augment the state representation when learning the forward looking agent. For example, a hint-feature can be the distance to the backward

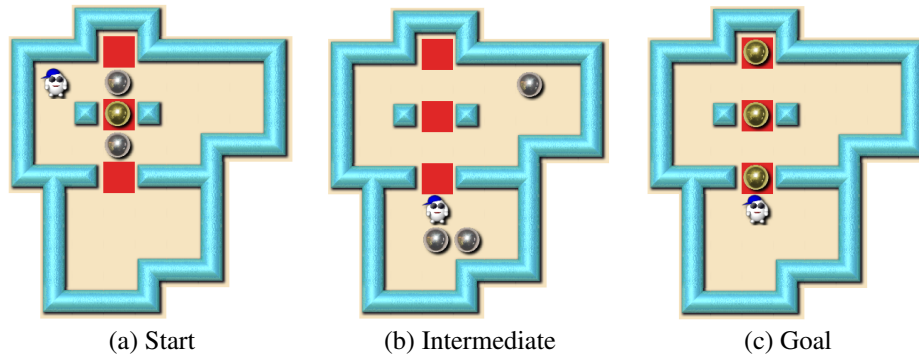


Figure 1: Illustration of the progression of the agent (guy with a hat) as boxes (circles) are moved in Sokoban level LOMA04-04. Note the need to first push away boxes from the targets (red squares) at the initial position (a), in order to clear a path (b) that ultimately allows the agent to pack all boxes into their final target positions (c). The strategies needed at the beginning and end phases are quite different.

plan. If this feature is minimized to zero then a solution has been found. Thus, this feature can serve as a useful signal for the forward facing agent. There are two important benefits to using these informed features instead of the policy of the backward agent directly. First, instead of imitating a specific (possibly sub-optimal) policy we allow the forward learning procedure to "discover" advantageous properties of the backward experiences via the learned value function. Second, this complete decoupling of the forward and backward policies allows us to also use relaxed backward facing agents that are given a much simpler but related goal. In the game of Sokoban, for example, the backward agent is simply required to pull all boxes away from the targets to any position on the board instead of the more difficult problem of moving them back into their original initial position. The ability to simplify the task of the backward agent can be quite important in cases where the backward task is as difficult as the forward one, and where simply walking backwards with a faraway reward will be as ineffective as walking forward.

Quite remarkably, using a small training set of just 155 beginner levels and simple RL and search algorithms, we are able to achieve state of the art results and solve 88 of the 90 benchmark XSokoban (Myers 1995) levels. To put this in perspective, without relying on the backward agent we solve only 60 levels, similarly to previously published works (Junghanns and Schaeffer 2001; Demaret, Van Lishout, and Gribomont 2008). In fact, using our looking back before looking forward RL approach, we do not only dramatically surpass other ML-based attempts but also improve the result of non-academic highly crafted systems. Importantly, as shown in the experimental evaluation section, the gains of incorporating features from the trajectories of the backward-facing agent are substantial both when using low-level features for training the forward looking agent as well as when relying on higher level ones.

Aside from the obvious contribution of making a substantial state of the art headway for the challenging game of Sokoban using learning, our contribution is in showing that by decoupling the policies of the forward and backward facing agents, we are able to make use of a simplified backward

agent. This, in turn, via simple *hint features*, facilitates improvement even in scenarios where the backward task is in itself quite difficult.

Related work

The idea of incorporating simulated or imagined experiences to aid a learning agent is an old one and underlies the Dyna-Q algorithm (Sutton 1990). Prioritized sweeping (Moore and Atkeson 1993) is an important refinement that introduces the idea of *backward focusing*, that is using simulated backward trajectories to prioritize exploration in the context of model-based RL. The Queue-Dyna algorithm (Peng and Williams 1998) introduces the same idea in the model free context of Q-learning. Both of these early approaches simulate backward moves from all states explored, and are not focused on faraway reward states which are hard to reach.

Closer to our work are the more recent works of (Edwards, Downs, and Davidson 2018) and (Goyal et al. 2019) which explicitly tackle the sparse reward scenario and initiate the backward agent at such states. Aside from various technical differences from our work such as the use of an explicit dynamics model for the backward agent, the key difference is that both approaches, either via data-augmentation or imitation, assume that the forward model can improve by mimicking the reverse policy of the backward agent. This may be true in domains that have start-end symmetries but is not generally so (see Figure 1 for an example). In contrast, we do not attempt to mimic the behavior or policy of the backward agent but instead introduce *hint features* that allow the forward agent to *learn* which properties of the backward trajectories are beneficial.

Many academic and practical Sokoban solvers have been developed. The milestone Rolling Stone solver (Junghanns and Schaeffer 2001) uses a heuristic function to lower bound the number of required pushes and, using IDA* search and several domain-specific enhancements, solves 59 of the 90 XSokoban levels. The solver has no backward search component, nor does it employ learning. Talking Stones (Demaret, Van Lishout, and Gribomont 2008) uses a different approach where the order in which the boxes should be packed is com-

puted first, followed by an attempt to carry it out, and solves 54/90 levels. The packing order is reminiscent of our backward search in that working from the goal state backwards we implicitly define such an order. However, unlike our fully learned approach, their computation of the packing order is hand-crafted and works only in certain types of levels. Further, our backward facing agent produces trajectories and a packing order is just one of several features that can be extracted from these simulated experiences.

Noteworthy non-academic solvers include Sokolution (Diedler 2017), Takaken (Takahashi 2008) and YASS (Damgaard 2000) which are highly hand-crafted and solve 81, 86, and 84 levels, respectively. Solving the full set of 90 levels remained an open problem for more than 20 years until it was recently solved by the FESS algorithm (Shoham and Schaeffer 2020). FESS relies heavily on powerful human crafted high-level features and heuristics. Our look back backward before forward approach is competitive and solves 88 levels using little training data, simple features and a fully learned value function.

The above works inherently rely on human hand crafting. More recently, DeepMind applied exciting breakthroughs in model-free reinforcement learning for Sokoban. Their Imagination-Augmented Agents (I2A) method (Racanière et al. 2017) performs rollouts within the evaluation function, thereby imagining the consequences of its decisions and planning ahead. Their model is able to learn the game using only low level features (pixels). Subsequent work goes a step further and shows that a general architecture (convLSTM) can exhibit many of the characteristics associated with planning (Guez et al. 2019). Although these works are indeed quite innovative, in the domain of Sokoban they were only applied to simplistic artificially generated levels with few boxes (called Boxoban), and made use of a large body of 900,000 training levels. Further, their own initial experiments indicate that performance quickly degrades as the number of boxes is increased. Our approach which uses a backward facing agent to generate *hint features* and trains on a mere 155 levels, easily solves all of these simple Boxoban levels as well as 88/90 of the challenging XSokoban ones.

Another recent work that backtracks from the goal introduces the idea of *reverse learning* or gradually increasing the difficulty of the learning task by pushing starting states further away from the goal (Florensa et al. 2017), training an increasingly more powerful agent at each iteration. While quite appealing, the approach relies on our ability to effectively sample nearby states all the way to the original ones. This can be effective in domains where the agent cannot get stuck, e.g. robotic manipulation, but will not work in scenarios where it is easy to reach dead-ends or deadlocks, as in the game of Sokoban.

That said, the above approach can be beneficial if used in a different setup. (Feng, Gomes, and Selman 2020) use this approach to quite effectively tackle difficult Sokoban levels by training a *level specific* model on easier variants of that level with less boxes. The setting is quite different than ours since the learning procedure is *given access to the test level* to be solved. Learning the model is thus actually part of the inference or solution process, with substantial run time

ramifications: while (Feng, Gomes, and Selman 2020) require 120 GPU hours to solve each new test level, our approach that generalizes across levels takes just few minutes on a single core to solve a previously unseen puzzle.

Reinforcement Learning Preliminaries

Reinforcement learning (RL) problems are characterized via a Markov Decision Process (MDP) that describes an environment in which actions take an agent from one state to the other, while reaping rewards or losses. $\mathcal{S}$ defines the collection of states s in the environment. At each state, the agent can take an action $a \in \mathcal{A}$, and stochastically transition into another state s' and receive a reward $r \in \mathcal{R}$, according to the transition probability $P(s', r | s, a)$. A policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ determines the action that the agent takes when in state s . Given P and π , the value of each state $V(s)$, that is the expected reward when starting in s and following the policy π , can be recursively computed.

Our goal in RL is to learn an effective policy, i.e., the policy $\pi^*(s)$ such that, starting from any state, the expected reward will be maximized. In value-based model-free RL, and in particular when the state space is large, instead of explicitly learning the transition function and using it to evaluate each action, we directly estimate the optimal value function $V^*(s)$ (or the action-value function $Q^*(s, a)$) from experienced or simulated agents trajectories $s_0, a_0, r_1, s_1, a_1, r_1, \dots$, and compute the optimal policy from optimal value function. That is, the task of learning becomes the task of computing the parameters of the value function $V^*(s)$. In large state-spaces, where a tabular representation of $V(s)$ is impractical, we often use an approximate value function representation. The simplest such approximation, which is the one we use in this work, is the linear approximation $V(s) \equiv \mathbf{w}^T \mathbf{f}(s)$, where $\mathbf{f}$ is a set of features computed from the state s , and $\mathbf{w}$ are the model’s parameters.

The approach we propose in this work can use any model-free RL method as a black-box procedure. In our application to Sokoban we use simple temporal difference, or $TD(0)$, one of the simplest yet quite effective of these methods. In $TD(0)$, given an experienced step $s_t, a_t, r_{t+1}, s_{t+1}$, the value function is updated according to the rule

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)],$$

where α is the learning or update rate that determines the velocity of the update rule, and γ is the standard optional discount factor. Appealingly, this gradient-like update rule converges to the optimal value function given mild conditions and sufficient experiences or episodes (a sequence of state/action/reward that the agent goes through). However, it also highlights the difficulty in the case of sparse rewards since r_{t+1} will be zero most of the time, and numerous experiences are needed for the reward to propagate backwards. See (Sutton and Barto 2018) for a detailed exposition of MDPs, model-free RL in general and TD learning in particular, and the concept of value function approximation.

Look Back Before Forward Reinforcement Learning

As discussed, our goal is to improve the learning of an agent when the signal of far-off sparse rewards is insufficient to facilitate effective learning. For simplicity of exposition we focus on the scenario where a single reward is given at the goal state, e.g., a manipulation task or a single player game. Intuitively, we can "narrow the gap" between the target and initial states by backtracking from the goal, and use this imagined experience to guide the learning process of the forward facing agent.

However, attempting to reverse-imitate the backward policy can be detrimental. In particular, unless the domain exhibits certain regularities, the effective policy near the goal can be quite different from the one needed at the initial state. To exemplify this, consider the simple Sokoban level shown in Figure 1. Starting from the near goal position (b), an agent would learn to decrease distances between boxes and targets. However, moving from the initial state (a) to the middle state (b) requires the opposite strategy. In our experimental evaluation section we demonstrate this quantitatively.

We suggest an alternative straightforward approach where, instead of trying to mimic the trajectories of the backward agent, we build simple *hint features* that relate states of these trajectories with states of the standard forward-facing agent. This allows the forward facing procedure to learn which aspects of the backward trajectories are useful for the forward policy and which are not, which in turn leads to substantial performance improvements. Below we describe the mechanics of the backward facing agent and the construction of the *hint features*, the learning of the forward facing agent, and inference or the solution of an unseen task. In the next section we make this generic approach concrete for the game of Sokoban.

The Backward Agent

Recall that we focus on scenarios where rewards are sparse, leading to slow convergence of RL algorithms, and requiring numerous training sequences. For simplicity, and w.l.o.g., let us assume that the rewards are only given at goal states $\mathcal{S}_R$, and denote by $\mathcal{S}_I$ the set of states where the agent can be initialized (in a typical game, this will be a single state where the player starts).

Similarly to all works that make use of backward moves (e.g. (Edwards, Downs, and Davidson 2018; Florensa et al. 2017; Goyal et al. 2019)), we assume that the agent can be initialized at the goal states $\mathcal{S}_R$ and that a backward task can be executed. With this setup in hand, the process of training the backward agent and the derivation of the backward trajectories are outlined in Algorithm 1 and described below.

Our backward agent can be learned using any standard model-free RL algorithm (we use simple $TD(0)$ in this work), but starting the agent in the target positions where rewards are received $\mathcal{S}_R$ instead of one of the initial positions $\mathcal{S}_I$. The first step is thus to design a reversed task via the construction of reversed rewards R_B . We then use these rewards to train an agent while taking backward steps.

It is important to note that in some cases (e.g. Sokoban)

Algorithm 1: TrainBackwardModel

Data: A set of training instances

Result: Backward value function $\mathcal{V}_B$

$R_B \leftarrow$ construct backward task rewards ;

$\mathcal{V}_B \leftarrow$ learning using standard RL with:

- (1) R_B as reward
- (2) Episodes initialized at goal states $\mathcal{S}_R$
- (3) Reverse moves

return $\mathcal{V}_B$

the backward problem can be as difficult as the forward one. In such case it can be useful to present an easier task for the backward agent by using different rewards. As an example, in a large maze navigation challenge, the agent may receive a reward by simply traversing half of the (aerial) distance toward the target. As we shall see in the experimental evaluation for the Sokoban game, trajectories of the backward agent, even when learning a relaxed problem, can be quite effective in providing useful signals to the forward agent.

Backward Trajectories

Having learned the value function of the backward agent $\mathcal{V}_B$, this model can now be used by both the forward learning procedure and the inference procedure below to generate backward trajectories from which useful *hint features* will be computed. A backward trajectory T for a particular (train or test) instance is created by:

1. Initializing the agent at goal states $\mathcal{S}_R$
2. Using a planning algorithm to find T that maximizes $\mathcal{V}_B$

Note that we retain a trajectory T even when the backward agent fails to achieve its full or relaxed goal, as even partial trajectories can be a useful guide to the forward facing agent.

The Forward Agent

With the backward agent's value function $\mathcal{V}_B$ in hand, we are now ready to leverage this in order to construct useful *hint features* and learn an effective agent whose goal is to solve the forward looking task.

The procedure is outlined in Algorithm 2 and explained below. We start by construction of backward trajectories (as described above) for each training instance. We then use any standard RL algorithm to learn a forward facing value function $\mathcal{V}_F$ (in our experiments we use simple $TD(0)$). The agent is initialized at $\mathcal{S}_I$ and with forward original rewards given at goal states $\mathcal{S}_R$.

The key difference from standard learning is that we use an augmented features space

$$f'(s) = f(s) \cup_i f_i(s, T)$$

where $f_i(s, T)$ are *hint features* computed based on the current state s and the instance specific backward trajectory T constructed using the backward agent $\mathcal{V}_B$. A simple but effective example of such a feature is the Manhattan distance between the final state reached by the backward agent and the current state of the forward agent. Intuitively, such features

Algorithm 2: TrainForwardModel

Data: Training instances and a backward model $\mathcal{V}_B$

Result: A forward value function $\mathcal{V}_F$

foreach *training instance m do*

$T_m \leftarrow$ backward trajectory for the instance
 constructed using $\mathcal{V}_B$

end

Use a standard RL algorithm to learn $\mathcal{V}_F$ with:

- (1) Forward rewards
- (2) Episodes initialized at initial state S_I
- (3) Forward moves
- (4) For each instance m , augmentation of the features for each state encountered with hint features $f_i(s, T_m)$

return $\mathcal{V}_F$

Algorithm 3: Inference

Data: An instance to be solved, value functions $\mathcal{V}_B$ (backward agent) and $\mathcal{V}_F$ (forward agent)

Result: A solution to the instance

$T \leftarrow$ backward trajectory for the instance constructed using $\mathcal{V}_B$;

Use a planning algorithm to maximize $\mathcal{V}_F$ with:

- (1) Start at S_I , forward rewards and moves
- (2) Augmentation of features for each state encountered with hint features $f_i(s, T)$

return A solution if any of S_G is reached

are informed (by the backward agent) hints that can point the forward agent in the right direction, both at learning and inference time. The forward learning procedure assigns the appropriate weights to these features, in the context of the original forward task.

Inference

When faced with a new previously unseen task, i.e. when performing inference, we rely on both the learned backward value function $\mathcal{V}_B$ and the forward value function $\mathcal{V}_F$. The simple inference procedure is outlined in Algorithm 3.

The backward agent is executed from the goal states and the resulting backward trajectory is used to construct hint features for the forward facing model. The forward agent is then applied to the original task at hand with these features and a solution is returned.

It is important to acknowledge that this seemingly generic approach requires some specification for the domain at hand: defining the relaxed backward task; designing backward rewards; constructing hint-features. At the same time, we do note that this domain specific tailoring can be quite straightforward, as discussed in the next section for the game of Sokoban. In our experimental evaluation, we demonstrate that even with this simple tailoring we can achieve SOTA

results.

Application to Sokoban

We now make our idea of looking back before forward RL concrete for the game of Sokoban, invented by Hiroyuki Imabayashi in 1981 and published by Thinking Rabbit. The player, represented in Figure 1 as a guy with a hat, can move around in all four directions and is able to push boxes one square ahead, if that space is vacant. The player cannot pull or push a box sideways. The goal in each level is to push all boxes to target positions, depicted as red squares. Most levels are meant to be challenging, creative and fun, and humans can solve them in a reasonable time. Theoretically, Sokoban has been shown to be NP-hard (Dor and Zwick 1999), and in practice it still poses a great challenge to automated solvers: it has a large branching factor, requires long solutions, and is riddled with irreversible moves that result in deadlocked states, e.g. when a box is pushed into a corner. We start by describing the value function representation, followed by a short exposition of the specifics of the forward and backward agents.

Value Function Representation

As discussed, we use the $TD(0)$ learning algorithm with a simple linear approximation for the representation of the value function. The core features used by both the forward and backward agents are:

- **Targets:** a count of the number of boxes already packed on target squares.
- **Distance:** the total number of pushes required to move all boxes onto target squares. This feature does not take into account the order in which the boxes should be pushed nor the fact that intermediate moves that push boxes away from targets may be required. Therefore, it is a lower bound on the actual number of moves needed.
- **Gamma1:** A crude estimate of the reward for solving the level, while taking reward discounting into account. Concretely, $\gamma_1 = \gamma^{\#boxes}$.
- **Gamma2:** A refined version of the Gamma1 estimate, taking into account the number of boxes that have already been packed. Concretely, $\gamma_2 = \gamma^{\#boxes - \#packed}$. For the backward search, the number of packed boxes is replaced by the number of unpacked ones.

The above features are all quite low-level in that they are only based on simple knowledge of the game rules and objective (packing of boxes), without any insight into how the game should be played. As we show in the experimental evaluation, these features are all that is needed to reach results that are quite impressive and that are already competitive with state-of-the-art performance. In our final solver we also use the following higher level feature:

- **Connectivity:** Sokoban positions can be divided into regions. In each region, the player can move freely. The player, however, cannot move to another region without first pushing boxes that block the path that connects the regions. For example, in Figure 1 (c) the connectivity is 2.

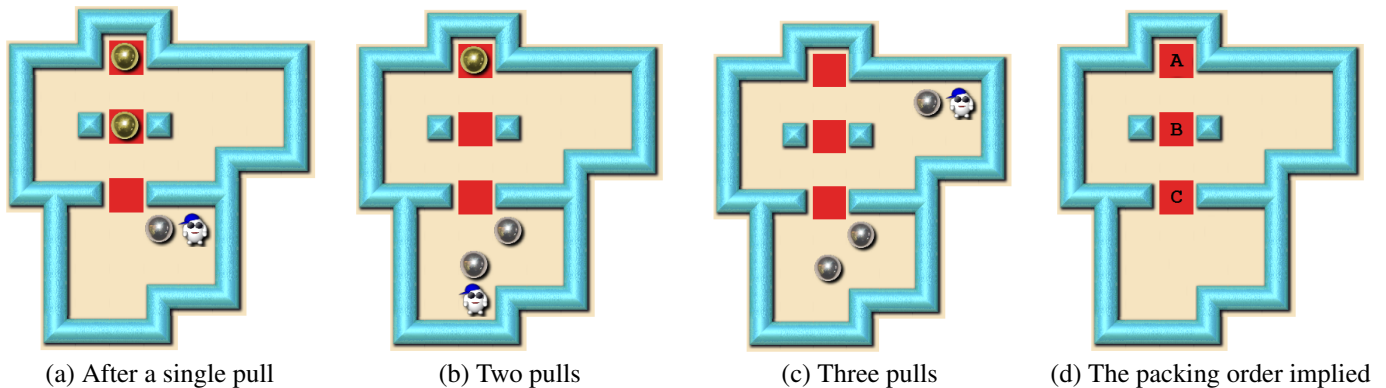


Figure 2: Illustration of the progress of the backward facing agent as it pulls boxes away from targets. (a) shows the position after pulling the bottom box away from the target and clearing the path for the agent to pull the middle box (b). Finally, the agent can clear the top box (c) and a top-middle-bottom packing order is implied (d).

All of the above features are normalized to the range $[0, 1]$. For example, one of five boxes on a target will result in a **Targets** value of 0.2. This normalization allows the model to generalize across levels of different difficulty, and keeps the gradients stable during training. Note that, despite this, the number of boxes in a level does affect the expected reward via the discounting factor γ .

Looking Backward Before Forward

The backward agent is initialized at the goal state, i.e., in a board where all the boxes are packed in target positions. The backward agent is trained to solve a relaxed problem and receives a reward as soon as *all* boxes are pulled away from the target squares to *anywhere* on the board, even if they are not back at their original position. Note that we do not use any reward shaping, and rely solely on the simplest possible reward. Indeed, tailoring of the backward task and its rewards requires only a rudimentary understanding of the puzzle.

Before describing the specifics of how we use the backward trajectories in the context of Sokoban, it is instructive to see an example progress of the backward agent as it starts in the final position and attempts to pull boxes away from targets as shown in Figure 2. Note that even positions (a) and (b), before the agent has managed to complete the task, reveal some configurations that could be useful hints for the forward search. Thus, such intermediate positions can also provide useful *hint features* that will point the forward facing agent in the right direction. Indeed, in complex levels our backward agent does not always succeed yet its trajectories still allow us to substantially improve the forward performance.

Given the trajectory of the backward agent, we augment the features set with two additional *hint features*:

- **Overlap.** A simple way of measuring similarity between board positions is to count the number of boxes which are on the same squares. The overlap feature is the maximum of this number measured between the current state s and all states of the backward trajectory $s_b \in T$. This is analogous to measuring the distance between a point and a curve. Note that the approach is general and does not rely on any Sokoban specific knowledge.

- **Perm.** This is a higher level feature that counts the number of boxes in the forward state s that are on target squares, but measured according to the packing order implied by the backward trajectory. In this trajectory, boxes are removed from targets in a specific order. Reversing this plan suggests an order in which boxes should be packed. See figure 2(d) for an example of a packing order.

All the features that we use are taken from existing Sokoban literature, with no further feature-engineering. The only exception is the **Overlap** hint-feature, which relates the backward trajectory to the forward search and is unique to our method. We do note, however, that this is a simplistic feature which only counts boxes as a naive measure of the distance between board positions. It does not contain any further inductive bias as to how the puzzle should be tackled.

Training Setup

Training set. We train both the forward and backward agents using only the 155 Microban levels (Skinner 2000) which have been developed as "a good set for beginners and children". The forward facing agent receives a reward of 1 when all boxes are on targets, and the backward facing agent receives a reward of 1 when it achieves the easier goal of unpacking all boxes away from the targets to any position on the board.

Search Tree. During both training and inference, as is standard when using RL for puzzle solving, a search tree is maintained. At each iteration a node in the search tree is expanded and its children are assigned a value based on the current value function and a reward, if received. The value of the best move serves as a new, better estimate for the position chosen for expansion, and the model weights are updated, as in standard $TD(0)$. The planning algorithm that we use for node expansion is a standard ϵ -greedy search: traversal of the tree begins at the root and descent is to the highest value child with probability $1 - \epsilon$, and uniformly over children otherwise. When a leaf is reached, it is expanded. Note that the learning is off-policy: the value function is updated using the best move, even when not picked by the ϵ -greedy policy.

When the construction of the backward search tree is complete, we pick the leaf with the highest value. The branch from the root to the leaf determines the backward trajectory.

Training epochs. In each iteration, the 155 training levels are randomly permuted and searched by the agent until a reward is received, with a cap of 50, 100 search nodes for the backward and forward agents, respectively. We perform 100 training iterations starting with a $TD(0)$ learning rate $\alpha=0.01$, and multiplying it by 0.98 at each iteration. Training takes a few minutes on a single core.

Experimental Evaluation

We now evaluate the merit of our look back before forward reinforcement learning approach using *hint features* on the challenging game of Sokoban.

Experimental Setup

As outlined in the previous section, we train on the simple 155 Microban levels (Skinner 2000). We evaluate performance on the 90 XSokoban levels (Myers 1995) which present a medium-level challenge to humans but that are still quite challenging to automated solvers.

As previously described (see Algorithm 3), given trained backward and forward agents, each test level is processed at inference time as follows. We first let the backward agent tackle it, starting from the goal state. We then use the trajectory of that agent to compute our *hint features*, and then apply the forward facing agent with these features. The exact mechanics of the agents are identical to the learning phase and described in the training setup part of the previous section. The backward search is capped at 10000 tree nodes, and the forward search at 50000, which corresponds to 5 minutes on a single core machine, with no special optimization.

We compare our solver to a baseline that is trained in an identical manner but without the *hint features* that are generated by the backward agent. We also compare to a baseline based on reverse curriculum learning (Florensa et al. 2017), adopted to Sokoban: we pull away $n/2$ boxes from the target, resulting in positions that are close to the target but not degenerate. We then augment the training data with simpler tasks by adding instances with these positions as the starting points.

Baselines for other Sokoban solvers are taken from the Sokoban solvers statistics site (Minklei 2009), which compares various solvers on several data sets. The site considers a level to be solved if a solution is found in less than 10 minutes, and our experiments meet this criteria.

XSokoban Results

The results are summarized in Figure 3. Using our look backward before forward approach (solid black line), we are able to solve 88 of the 90 XSokoban levels. This surpasses the highly crafted solvers Sokolution (Diedler 2017), Takaken (Takahashi 2008) and YASS (Damgaard 2000) that can only solve 81, 86, 84 levels, respectively. Our result is second only to the FESS solver (Shoham and Schaeffer 2020). Impressively, we are able to do so while training on only 155 simple levels and using straightforward RL techniques.

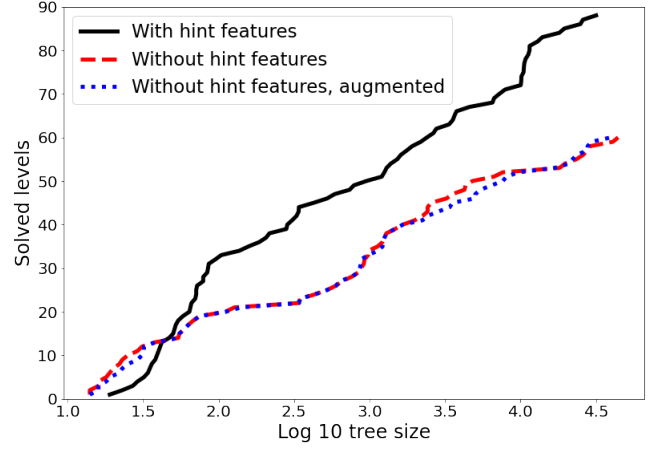


Figure 3: Summary performance on the 90 XSokoban test levels. The graph shows the number of solved levels as a function of the search tree size at inference time. Compared are our method (solid black), a baseline without *hint features* (dashed red), and the baseline augmented with near-goal trajectories (dotted blue).

	Overlap	Perm	Connect	Solved
No Hints	no	no	no	55
	no	no	yes	60
With Hints	yes	no	no	82
	yes	no	yes	86
	yes	yes	no	86
	yes	yes	yes	88

Table 1: Summary performance on the 90 test XSokoban levels for different combination of features. Shown is the number of levels solved for each variant. Note the substantial gap between the standard **No Hints** learning setup and our **With Hints** approach.

The added value of the *hint features* is obvious and without them we are only able to solve 60 levels. Further, augmenting the experiences of the forward solver with near-goal trajectories leads only to a minor improvement. This is due to the fact that the backward and forward setting can be quite different, emphasizing the importance of *deriving* features from the backward agent rather than trying to mimic it.

It is natural to wonder how the effectiveness of the *hint features* depends on the quality of the features used. Specifically, whether higher level features such as **Connectivity** or **Perm** that arise from an understanding of the game are needed for our approach to be useful. To evaluate this, we also tried learning forward and backward agents with only the most basic features, i.e. without connectivity and using only the overlap count as a *hint feature*. As can be seen in Table 1, even the weakest look back before looking forward variant is able to solve 82 levels, already putting it on par with the best existing solvers, highlighting the power of *hint features* even in simple settings.

Finally, it is insightful to look at the weights of the linear

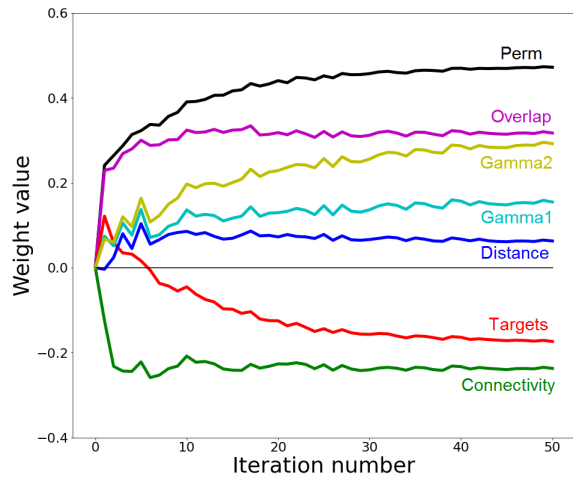


Figure 4: The weights of the linear value function V_F as it progresses during the learning iterations of our looking back before forward approach.

approximation value function. These are shown in Figure 4, as they progress during learning iterations. The weight of **Connectivity** (green) is negative, as expected. The weight of **Targets** (red) starts out as positive as the agent gets its first rewards. The agent then learns that moving boxes to targets is detrimental if this does not follow the packing plan, and **Targets** drifts to a negative value. The positivity of **Distance** (blue) might be surprising for those who have not played Sokoban but indeed one often needs to first push boxes away from the targets in order to clear the path for the final packing, as is the case in the example of Figure 1. Most appealingly, as might be expected from the quantitative results above, the *hint features* **Perm** (black) and **Overlap** (purple) play a dominant role.

Summary

We tackled the task of learning an effective solver agent for the challenging Sokoban puzzle where the reward is singular and given only at the goal state. We presented a novel approach that first solves a (possibly relaxed) reverse task, and then uses this solution to construct *hint features* that help guide the forward facing agent during both training and inference time. Using our approach we were able to solve 88 of the 90 XSokoban levels, a near perfect result. Further, to the best of our knowledge, although RL has been applied to simplistic Sokoban levels, ours is the first work that is based on machine learning and generalizes across levels to tackle the standard XSokoban benchmark set.

Importantly, using our look back before forward RL approach, we were able to achieve this state of the art result while training on a small set of just 155 practice levels, a naive linear function approximation, and standard RL techniques. Further, we demonstrated that we can achieve near state of the art results even with simple features that do not rely on an understanding of how the game should be played. We also observed that the value function learned by our method captures non-trivial strategies such as the need to push blocking boxes

away from targets before executing a final packing order that allows our agent to reach its goal.

Our work was motivated by sparse reward scenarios where we do not have access to a large number of training instances, and are thus not able to use low-level methods that are sample-heavy (e.g., (Mnih et al. 2015; Silver et al. 2016; Guez et al. 2019)). Indeed, in our early experiments, such methods did poorly on XSokoban levels, which is not surprising given the small number of simple training levels available. In future work, it would be useful to explore a fusion of these works and our looking back approach in middle-ground scenarios where more training data is available but still not abundant.

References

- Damgaard, B. 2000. YASS Sokoban solver. Available in <https://sourceforge.net/projects/sokobanyasc/>.
- Demaret, J.-N.; Van Lishout, F.; and Gribomont, P. 2008. Hierarchical planning and learning for automatic solving of sokoban problems. *Belgian/Netherlands Artificial Intelligence Conference* 57–64.
- Diedler, F. 2017. Sokolution Sokoban solver. Available in <http://codeanalysis.fr/sokoban/>.
- Dor, D.; and Zwick, U. 1999. SOKOBAN and other motion planning problems. *Computational Geometry* 13: 215–228.
- Edwards, A. D.; Downs, L.; and Davidson, J. C. 2018. Forward-Backward Reinforcement Learning. *CoRR*.
- Feng, D.; Gomes, C.; and Selman, B. 2020. Solving Hard AI Planning Instances Using Curriculum-Driven Deep Reinforcement Learning. In Bessiere, C., ed., *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*, 2198–2205. International Joint Conferences on Artificial Intelligence Organization. doi: 10.24963/ijcai.2020/304. URL <https://doi.org/10.24963/ijcai.2020/304>. Main track.
- Florensa, C.; Held, D.; Wulfmeier, M.; Zhang, M.; and Abbeel, P. 2017. Reverse Curriculum Generation for Reinforcement Learning. In Levine, S.; Vanhoucke, V.; and Goldberg, K., eds., *Proceedings of the 1st Annual Conference on Robot Learning*, 482–495.
- Goyal, A.; Brakel, P.; Fedus, L.; Singhal, S.; Lillicrap, T.; Levine, S.; Larochelle, H.; and Bengio, Y. 2019. Recall Traces: Backtracking Models for Efficient Reinforcement Learning. In *International Conference on Learning Representations*.
- Guez, A.; Mirza, M.; Gregor, K.; Kabra, R.; Racaniere, S.; Weber, T.; Raposo, D.; Santoro, A.; Orseau, L.; Eccles, T.; Wayne, G.; Silver, D.; and Lillicrap, T. 2019. An Investigation of Model-Free Planning. In Chaudhuri, K.; and Salakhutdinov, R., eds., *Proceedings of the 36th International Conference on Machine Learning*.
- Junghanns, A.; and Schaeffer, J. 2001. Sokoban: Enhancing General Single-Agent Search Methods Using Domain Knowledge. *Artificial Intelligence* 129(1–2): 219–251.
- Minklei, F. 2009. Sokoban solver statistics. Available in http://sokobano.de/wiki/index.php?title=Solver_Statistics/.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; and Hassabis, D. 2015. Human-level control through deep reinforcement learning. *Nature* 518: 529–533.

Moore, A. W.; and Atkeson, C. G. 1993. Prioritized Sweeping: Reinforcement Learning with Less Data and Less Time. *Machine Learning* 13(1): 103–130.

Myers, A. 1995. XSokoban home page. Available in <http://www.cs.cornell.edu/andru/xsokoban.html>.

Peng, J.; and Williams, R. 1998. Efficient Learning and Planning Within the Dyna Framework. *Adaptive Behavior* 1.

Racanière, S.; Weber, T.; Reichert, D. P.; Buesing, L.; Guez, A.; Rezende, D.; Badia, A. P.; Vinyals, O.; Heess, N.; Li, Y.; Pascanu, R.; Battaglia, P.; Hassabis, D.; Silver, D.; and Wierstra, D. 2017. Imagination-Augmented Agents for Deep Reinforcement Learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*.

Shoham, Y.; and Schaeffer, J. 2020. The FESS Algorithm: A Feature Based Approach to Single-Agent Search. In *2020 IEEE Conference on Games (CoG)*, 96–103. doi:10.1109/CoG47356.2020.9231929.

Silver, D.; Huang, A.; Maddison, C. J.; Guez, A.; Sifre, L.; van den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; Dieleman, S.; Grewe, D.; Nham, J.; Kalchbrenner, N.; Sutskever, I.; Lillicrap, T.; Leach, M.; Kavukcuoglu, K.; Graepel, T.; and Hassabis, D. 2016. Mastering the Game of Go with Deep Neural Networks and Tree Search. *Nature* 529(7587): 484–489. ISSN 0028-0836. doi:10.1038/nature16961.

Skinner, D. 2000. Microban puzzle pack. Available in <https://www.sokobanonline.com/play/web-archive/david-w-skinner>.

Sutton, R. 1990. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *The Seventh International Conference on Machine Learning*.

Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. The MIT Press.

Takahashi, K. 2008. Takaken Sokoban solver. Available in <http://www.ic-net.or.jp/home/takaken/e/soko/>.